Kanton St.Gallen
Gewerbliches Berufs- und Weiterbildungszentrum St.Gallen

Grundbildung

IMS-T
gbssg.ch

Projektdokumentation

IMST@GBSSG

SmartClock

Sai Ragavan
Ausgabe v1.0

ISO 29990:2010 zertifiziert

Inhalt.

1 IPERKA. 3

1.1 Informieren 3

1.1.1 Komponenten Beschreibung 3
1.1.2 Ab wann ist die Luftqualität schlecht? 5

1.2 Planen 6

1.2.1 Verkabelungen 6
1.2.2 Planung der Programmierung 7
1.2.3 Menüs 9

1.3 Entscheiden 10

1.3.1 Stückliste 10

1.4 Realisieren 11

1.4.1 Zusammenbau 11
1.4.2 Programmierung 12

2 Benutzerhandbuch 19

2.1 Installation 19

2.2 Allgemeine Steuerungen 19

2.2.1 Knopf Steuerung 19
2.2.2 Joystick Steuerung 19

2.3 Wichtige Informationen 20

2.3.1 Sensoren 20
2.3.2 Joystick Verbindung 20
2.3.3 WLAN-Verbindung 20

3 Verzeichnis 21

3.1 Abbildung 21

1 IPERKA.

1.1 Informieren

1.1.1 Komponenten Beschreibung

Abbildung 1 - MicroMod ESP32

1.1.1.1 MicroMod ESP32

Das SparkFun MicroMod ESP32 ist ein Mikrocontroller, der programmierbar ist. Da man mit dem Chip allein, schwer mehrere Funktionen einbinden kann, lässt es sich einfach mit einem MicroMod Carrier Board verbinden, das zusätzliche Anschlüsse zur Verfügung stellt.

Abbildung 2 - MicroMod ATP Carrier Board

1.1.1.2 MicroMod ATP Carrier Board

Das SparkFun MicroMod ATP Carrier Board ist eine Erweiterungsplatte für den Mikrocontroller. Es besitzt zusätzliche Anschlüsse, um das Verbinden von weiteren Komponenten zu erleichtern. Mit Qwiic ermöglicht es eine einfache Verbindung.

Abbildung 3 - Environmental Combo Breakout

1.1.1.3 Environmental Combo Breakout

Das Environmental Combo Breakout Board kombiniert die Sensoren ENS160 für die Messung der Luftqualität (CO_2) und den BME280 für Temperatur, Luftfeuchtigkeit und Luftdruck. Dieses Board lässt sich einfach mit der Qwiic-Schnittstelle verbinden.

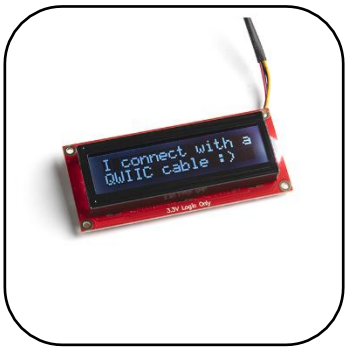

Abbildung 4 - SerLCD

1.1.1.4 Sparkfun 16x2 SerLCD

Das SparkFun 16x2 SerLCD ist ein LCD-Display mit 16 Zeichen pro Zeile, mit 2 Zeilen. Es hat ein schwarzes Display mit RGB-Hintergrundbeleuchtung, die angepasst werden kann. Auch dieses Display lässt sich leicht mit den Qwiic-Schnittstellen integrieren.

Abbildung 5 - Qwiic Button

1.1.1.5 Qwiic Button

Das SparkFun Qwiic Button ist ein Eingabegerät mit einem Knopf. Es hat ein LED, um ein Feedback zu geben. Die Verbindung ist mit Qwiic möglich. Dieses Komponent, wird verwendet, um die Anzeige auf dem Display zu ändern.

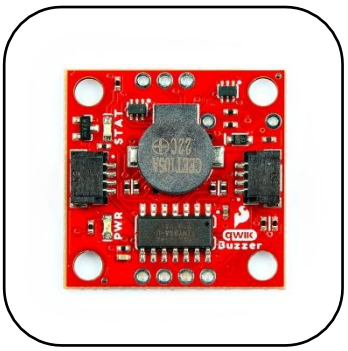

Abbildung 6 - Sparkfun Buzzer

1.1.1.6 Sparkfun Buzzer

Das SparkFun Buzzer ist Komponent, die akustische Alarme lässt. Dieses Komponent wird benötigt, um akustische Signale sowohl für Timer-Alarmfunktionen als auch für die Warnung von schlechter Luftqualität benötigen.

Abbildung 7 - Qwiic Cable

1.1.1.7 Qwiic Cable

Das SparkFun Qwiic Cable 50mm / 100mm ist ein Verbindungskabel mit Qwiic-Stecker an beiden Enden. Es ermöglicht eine Verbindung zwischen zwei Qwiic-Schnittstellen. Für mein Projekt ist diese Verbindung passend, da es sehr einfach ist zu verwenden und die Kommunikation zuverlässig ist.

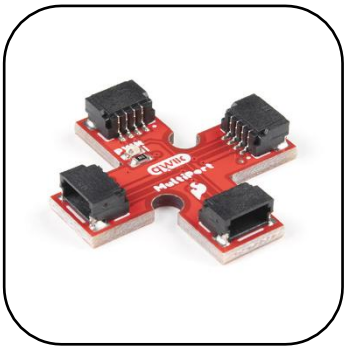

Abbildung 8 - Multiport

1.1.1.8 Sparkfun MultiPort

Das SparkFun MultiPort ist ein Erweiterungs-Komponent, welches ermöglicht, mehrere Geräte über die Qwiic-Schnittstelle an einen Mikrocontroller anzuschliessen. Dieses Komponent, ist sehr wichtig, da ich sonst Probleme hätte, alle Komponente zu verbinden.

Abbildung 9 - Thumb Slide Joystick

1.1.1.9 Thumb Slide Joystick
Der Joystick ermöglich eine grosse Anzahl von verschiedenen Inputs. Die Verbindung ist mit dem Löten und mit Jumper Wire möglich. Für das Einstellen des Timers ist dieses Komponent sehr hilfreich.

Abbildung 10 - USB-a zu USB-c

1.1.1.10 USB-a zu USB-c Kabel
Dieses Kabel wird für die Stromversorgung des MicroMods verwendet.

1.1.2 Ab wann ist die Luftqualität schlecht?

Die Luftqualität in Innenräumen wird oft durch den CO_2-Gehalt gemessen. Ein CO_2-Wert unter 400 ppm gilt als normal und bietet eine gute Luftqualität. Werte zwischen 400 und 1000 ppm sind akzeptabel, während ein CO_2-Gehalt über 1000 ppm als schlecht sind. Ab 2000 ppm kann die Luftqualität zu Problemen wie Müdigkeit und Konzentrationsstörungen führen. Eine regelmässige Belüftung ist wichtig, um den CO_2-Gehalt zu kontrollieren.

0 ppm – 400 ppm	Gute	
400 ppm – 1000 ppm	Akzeptabel	
1000+ ppm	Schlecht	

1.2 Planen

1.2.1 Verkabelungen

1.2.1.1 Erste Version

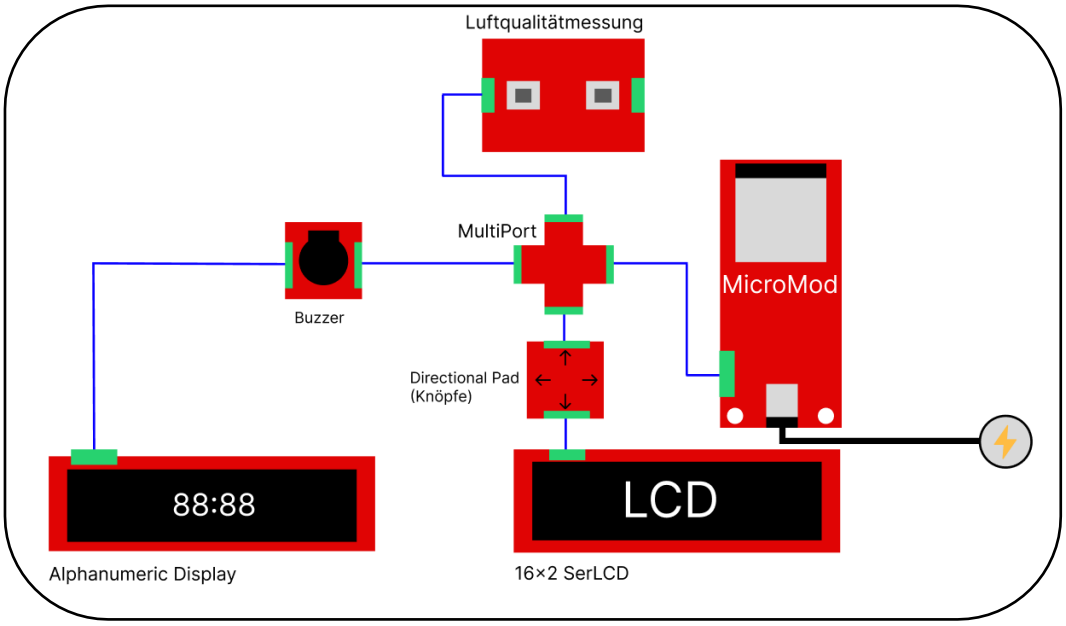

Abbildung 11 - Verkabelung V1

Mit dieser Version habe ich die Verkabelungen der Komponente festgestellt. Jedoch ist mir ein Fehler überflogen. Der MicroMod hat selbst keine Qwiic-Schnittstellen.

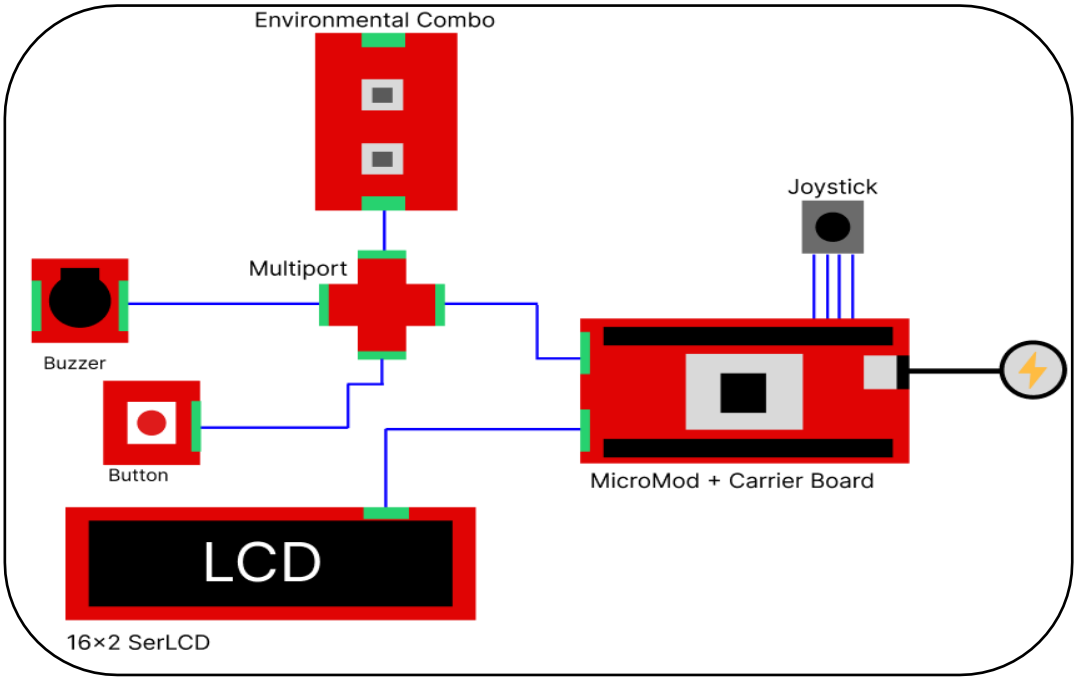

Abbildung 12 - Verkabelung V2

1.2.1.2 Zweite Version

Bei der Zweiten Version habe ich den Fehlenden Komponenten eingefügt. Da ich aber den Alphanumeric Display nicht benötigen werde, habe ich diesen entfernt. Zusätzlich habe ich den Directional Pad mit einen Joystick ersetzt und einen Knopf hinzugefügt.

1.2.2 Planung der Programmierung

1.2.2.1 Flowchart WarnBuzz

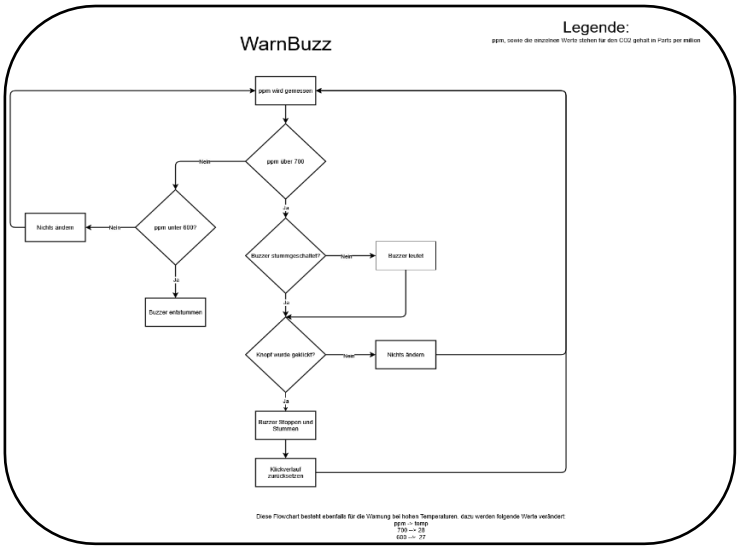

Abbildung 13 - WarnBuzz V1

Version 1
Wie bereits erwähnt ist dieses Flowchart für den Buzzer. Genauer gesagt, beschreibt dieses
Flowchart, wann der Buzzer läuten soll und wann nicht.

Hier wird überprüft, ob der CO2 Gehalt zu hoch ist, falls ja, wird zusätzlich geprüft, ob der Buzzer Stummgeschalten ist. Je nach Antwort klingelt der Buzzer oder eben nicht.

Wenn der CO2 Gehalt unter 600 liegt, wird der Buzzer entstummt und ermöglich den Buzzer bei Hohem Gehalt zu läuten.

Danach wird geprüft, ob der Knopf geklickt wurde. Dieser Schritt ist notwendig, für das Ausschalten und Stummen des Buzzers. Nebenbei wird noch der Klickverlauf gelöscht, da sonst der Knopf immer «geklickt» wird.

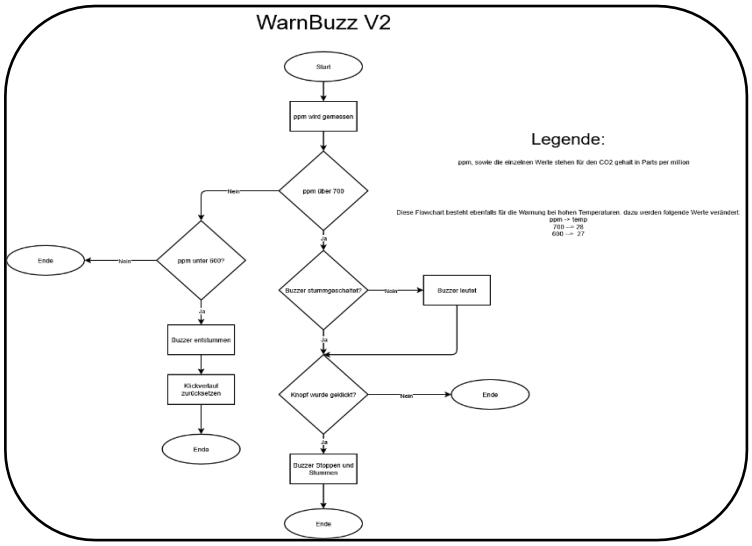

Abbildung 14 - WarnBuzz V2

Version 2
In der zweiten Version wurde der Startpunkt und die Endpunkte klarer definiert. Ausserdem wird der Klickverlauf bei einem tieferen Wert als 600 gelöscht und nicht bei jedem Knopf klick, wie zuvor.

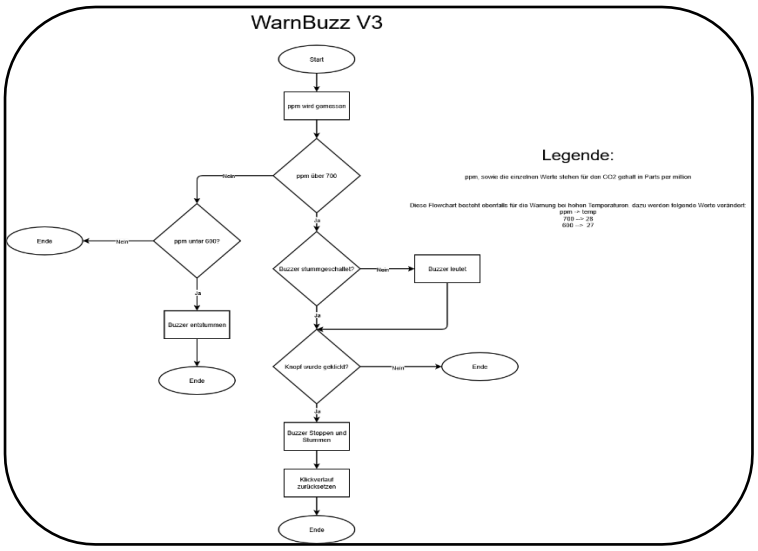

Abbildung 15 - WarnBuzz V3

Version 3
Es gibt nur einen Unterschied zwischen der zweiten und der dritten Version. Das ist das Löschen des Klickverlaufes. Das wird wieder bei jedem Klick des Knopfes ausgelöst.

1.2.2.2 *Flowchart SetTimer*

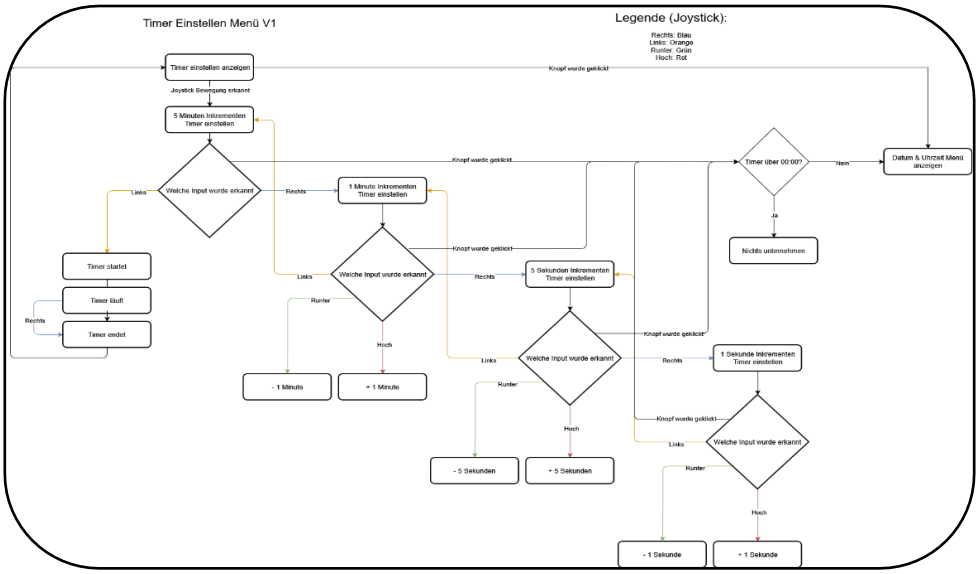

Abbildung 16 - SetTimer V1

Version 1
In diesem Flowchart wird die beschrieben wie das Menü für den Timer geöffnet und geschlossen wird, sowie die Anleitung für das Starten, Einstellen und Stoppen des Timers.

Diese Funktionen werden durch verschiedene Joystick Bewegungen und auch Kombinationen ausgeführt.

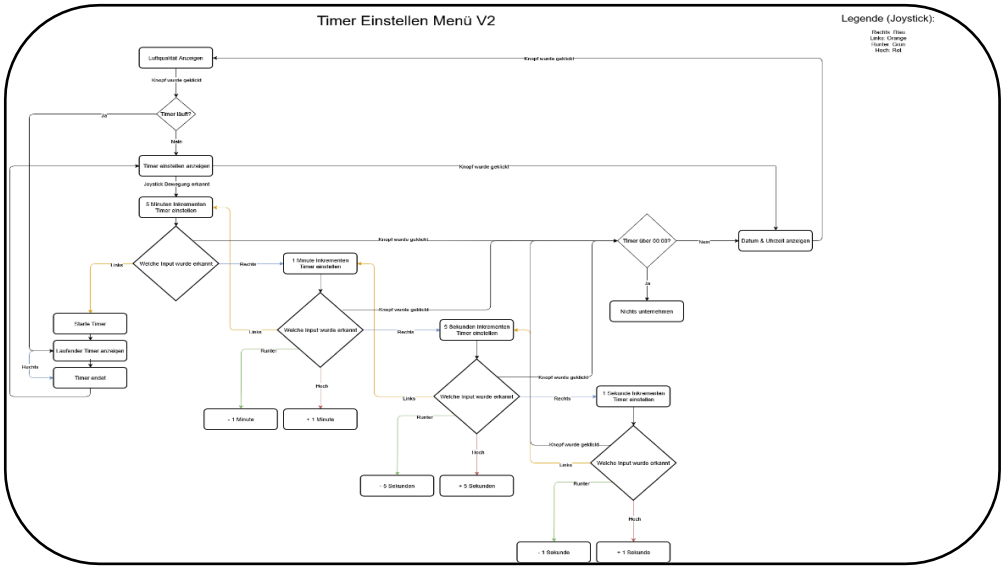

Abbildung 17 - SetTimer V2

Version 2
In der zweiten Version wurde die Logik des Timers nicht verändert. Es wird jedoch beim Öffnen des Timer Menüs geprüft, ob schon ein Timer läuft.

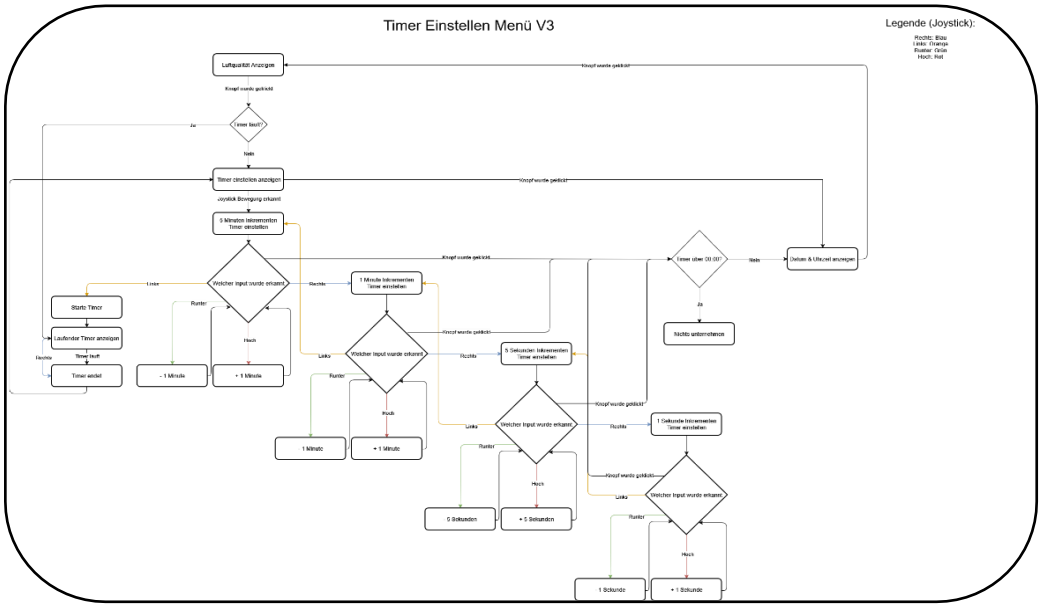

Abbildung 18 - SetTimer V3

Version 3
Da in der ersten und in der zweiten Version die Einstellung des Timers im Minuten Takt fehlt, wurde das noch in der dritten Version nachgearbeitet. Auch hier gibt es sonst keine Veränderungen in der Logik.

1.2.3 Menüs

1.2.3.1 Luftqualität Menü

Abbildung 19 - Luftqualität

Im Luftqualitätsmenü werden die Temperatur, der CO_2-Gehalt sowie die Luftfeuchtigkeit angezeigt. Durch Drücken des mittleren Knopfs öffnet sich das Menü für Datum und Uhrzeit. Wird der rechte Knopf gedrückt, gelangt man zum Timer-Menü.

1.2.3.2 Datum & Uhrzeit Menü

Abbildung 20 - Datum & Uhrzeit

Im Menü für Datum und Uhrzeit werden die aktuelle Uhrzeit in Stunden, Minuten und Sekunden sowie der Wochentag und das Datum angezeigt. Durch Drücken des rechten Knopfs öffnet sich das Timer Menü, während der linke Knopf zum Luftqualitätsmenü führt.

1.2.3.3 Timer Menü

Abbildung 21 - Timer

Im Timer-Menü wird die verbleibende Zeit bis zum Ablauf des Timers angezeigt. Der Timer kann über die Tasten eingestellt werden. Mit dem linken Knopf öffnet sich das Luftqualitätsmenü, der mittlere Knopf führt zum Menü für Datum und Uhrzeit.

1.3 Entscheiden

1.3.1 Stückliste

Nach langer Recherche habe ich diese Stückliste erstellt.

Anzahl	Komponente	Lieferant	Lieferantartikelnummer	Herstellerartikelnummer	Link	Preis in CHF Pro Stück	Preis Total in CHF	Begründung
1	MicroMod ESP32	Digikey	1568-16781-ND	WRL-16781	https://www.digikey.c	16.70	16.70	Mikrocontroller
1	Environmental Combo	Digikey	1568-22858-ND	SEN-22858	https://www.digikey.c	35.70	35.70	Sensoren für die Messung der Luftqualität
1	Qwiic LCD	Digikey	1568-16397-ND	LCD-16397	https://www.digikey.c	29.30	29.30	Display (Datum, Uhrzeit, Messungen)
1	Qwiic Directional Pad	SparkFun	PRT-26851	PRT-26851	https://www.sparkfun	6.10	6.10	Taster zur Anwendung der Funktionen
1	Qwiic Buzzer	Digikey	1568-24474-ND	BOB-24474	https://www.digikey.c	8.80	8.80	Akustische Warnung
4	Qwiic Kabel - 50mm	Digikey	1568-1710-ND	PRT-14426	https://www.digikey.c	3.20	12.80	Verkabelung der Komponente
1	Qwiic Kabel - 100mm	Digikey	1568-1711-ND	PRT-14427	https://www.digikey.c	1.60	1.60	Verkabelung der Komponente
1	Qwiic MultiPort	Digikey	1568-18012-ND	BOB-18012	https://www.digikey.c	2.60	2.60	Zusätzliche Schnittstellen zur Verbindung
1	ATP Carrier Board	Digikey	1568-16885-ND	DEV-16885	https://www.digikey.c	17.00	17.00	Erweiterungsboard (Schnittstellen)
1	Ladestecker (Adapter)	Digikey	102-3423-ND	SWI5-5-E-I38	https://www.digikey.c	6.00	6.00	Stromversorgung
1	USB A - USB A	Digikey	2223-CBL-UA-UA-05BP-ND	CBL-UA-UA-05BP	https://www.digikey.c	2.75	2.75	Stromversorgung
14							139.35	

Abbildung 22 - Stückliste V1

Durch weiterer Recherche habe ich die Stückliste überarbeitet.

Anzahl	Komponente	Lieferant	Lieferantartikelnummer	Herstellerartikelnummer	Link	Preis in CHF Pro Stück	Preis Total in CHF	Begründung
1	MicroMod ESP32	Digikey	1568-16781-ND	WRL-16781	https://www.digikey.	16.70	16.70	Mikrocontroller
1	Environmental Combo	Digikey	1568-22858-ND	SEN-22858	https://www.digikey.	35.70	35.70	Sensoren für die Messung der Luftqualität
1	Qwiic LCD	Digikey	1568-16397-ND	LCD-16397	https://www.digikey.	29.30	29.30	Display (Datum, Uhrzeit, Messungen)
1	Thumb Slide Joystick	Digikey	1568-09426-ND	9426	https://www.digikey.	5.00	5.00	Joystick zur Anwendung der Funktion
1	Qwiic Buzzer	Digikey	1568-24474-ND	BOB-24474	https://www.digikey.	8.80	8.80	Akustische Warnung
4	Qwiic Kabel - 50mm	Digikey	1568-1710-ND	PRT-14426	https://www.digikey.	3.20	12.80	Verkabelung der Komponente
1	Qwiic Kabel - 100mm	Digikey	1568-1711-ND	PRT-14427	https://www.digikey.	1.60	1.60	Verkabelung der Komponente
1	Qwiic MultiPort	Digikey	1568-18012-ND	BOB-18012	https://www.digikey.	2.60	2.60	Zusätzliche Schnittstellen zur Verbindung
1	ATP Carrier Board	Digikey	1568-16885-ND	DEV-16885	https://www.digikey.	17.00	17.00	Erweiterungsboard (Schnittstellen)
1	Ladestecker (Adapter)	Digikey	102-3423-ND	SWI5-5-E-I38	https://www.digikey.	6.00	6.00	Stromversorgung
1	USB A - USB A	Digikey	2223-CBL-UA-UA-05BP-ND	CBL-UA-UA-05BP	https://www.digikey.	2.75	2.75	Stromversorgung
1	Qwiic Button	Digikey	1568-15932-ND	15932	https://www.digikey.	4.65	4.65	Taster zur Anwendung der Funktionen
15							142.90	

Abbildung 23 - Stückliste V2

1.4 Realisieren

1.4.1 Zusammenbau

Als ich alle benötigten Komponenten bekommen habe, habe ich erst überprüft, ob alle Komponente funktionieren und nicht beschädigt sind. Danach habe ich ein Holzbrett bekommen, auf dem ich die Komponenten einzeichnen und ihre Verbindungen markieren konnte.

Nachdem die Skizze fertig war, konnte ich die Komponenten darauf festschrauben. Am Ende sah das dann so aus:

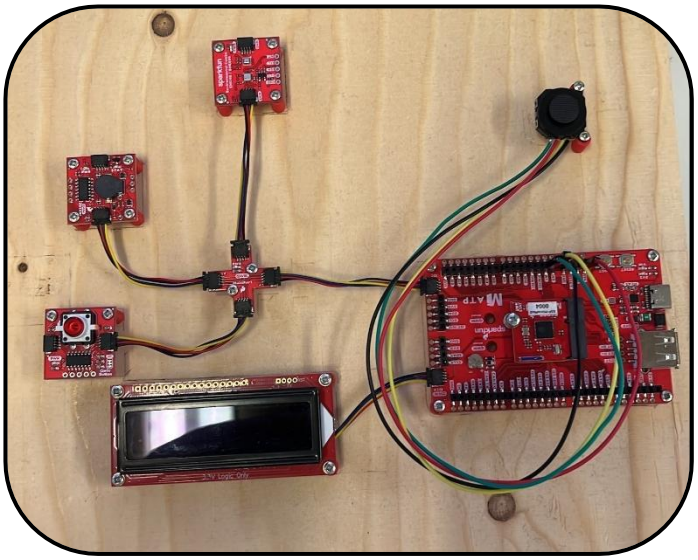

Abbildung 24 - Zusammenbau

Auf dem Holzbrett habe ich den LCD, den MicroMod mit dem ATP Carrier Board, den Knopf, die Sensoren, den Buzzer und den Joystick angeschraubt. Die Sensoren, der Knopf und der Buzzer werden mit Hilfe des MultiPorts verbunden.

1.4.2 Programmierung

Ich habe verschiedene Dateien für die Funktionen erstellt, die ich benötige, hier ist die Dokumentation vom geschriebenem Code.

1.4.2.1 LCD

```
void lcdSetup()
{
  Wire1.begin();
  lcd.begin(Wire1);
  lcd.setAddress(0x72);
  lcd.clear();
  lcd.setContrast(50);
  displayTimer.start(100);

  lcd.createChar(0, smiley);
  lcd.createChar(1, neutral);
  lcd.createChar(2, frownie);
  lcd.createChar(3, death);
  lcd.createChar(4, arrow);
  lcd.createChar(5, start);
}
```

Abbildung 25 - lcdSetup()

Die Funktion lcdSetup() stellt die Verbindung mit dem Mainboard da. Als erstes wird der LCD gestartet und es wird mithilfe von .setAddress(0x72) definiert wie der LCD kommunizieren soll. Danach wird die Anzeige noch zurückgesetzt bzw. geleert und es wird ein bestimmter Kontrast gesetzt, damit die einzelnen Zeichen gut erkennbar sind. Zusätzlich werden noch Grafiken erstellt, welche im späteren verwendet werden.

```
void printTimeAndDate()
{
  if (displayTimer.isTimeout())
  {
    lcd.setCursor(0, 0);
    lcd.print("Time:   ");
    lcd.print(timeClient.getFormattedTime());

    lcd.setCursor(0, 1);
    lcd.print("Date:   ");
    lcd.print(timeClient.getFormattedDate());
    displayTimer.restart();
  }
}
```

Abbildung 26 - printTimeAndDate()

Die Funktion printTimeAndDate() wird benötigt für die Anzeige des Datums und der Uhrzeit. Diese Anzeige ist funktioniert folgendermassen: Als erstes wird die Position vom ersten Zeichen bestummen. Da steht dann «Time: » folgend von der Formatierten Zeit, welche mit Hilfe von der NTPClient und Network Library aufgerufen werden kann. Dasselbe passiert mit dem Datum, auf der zweiten Zeile. Der Code befindet sich in der If-Schleife, damit diese Effizient, alle 100 ms ausgeführt wird. Die Dauer der Pause wird mit SimpleSoftTimer displayTime(Dauer) Deklariert.

```
void printTempAndCO2()
{
  if (displayTimer.isTimeout())
  {

    lcd.setCursor(0, 0);
    lcd.print("CO2 :");

    if (ppm >= highPPM)
    {
      lcd.print(ppm);
      lcd.print(" ppm ");
      lcd.setCursor(15, 0);
      lcd.writeChar(2);
    }
    else if (ppm < highPPM && ppm > midPPM)
    {
      lcd.print(" ");
      lcd.print(ppm);
      lcd.print(" ppm ");
      lcd.setCursor(15, 0);
      lcd.writeChar(1);
    }
    else if (ppm <= midPPM)
    {
      lcd.print(" ");
      lcd.print(ppm);
      lcd.print(" ppm ");
      lcd.setCursor(15, 0);
      lcd.writeChar(0);
    }

    lcd.setCursor(0, 1);
    lcd.print("Temp:  ");
    lcd.print(temp, 1);
    lcd.setCursor(12, 1);
    lcd.write(223);
    lcd.print("C");

    displayTimer.restart();
  }
}
```

Abbildung 27 - printTempandCO2

Die Funktion printTempAndCO2() hat viele Ähnlichkeiten mit der Funktion printTimeAndDate(). Es funktioniert mit der gleichen Logik. Die Werte werden mit den Sensoren aufgerufen und der Variable ppm bzw. temp gegeben. Bei bestimmten Werten werden auch passende Emojis angezeigt.

1.4.2.2 Buzzer

```
void warnBuzz()
{
  if (ppm > midPPM)
  {
    if (!buzzerMuted)
    {
      buzzerBuzzing = true;
      buzzer.configureBuzzer(2730, 1000, SFE_QWIIC_BUZZER_VOLUME_MIN);
      buzzer.on();
      buzzerMuted = false;
      currentState = AIR_QUALITY_STATE;
    }

    if (button.isPressed())
    {
      buzzer.off();
      buzzerBuzzing = false;
      buzzerMuted = true;
    }
  }
  else if (ppm < (midPPM - 100))
  {
    buzzerBuzzing = false;
    buzzerMuted = false;
  }
}
```

Abbildung 28 - warnBuzz()

Die Funktion warnBuzz() gibt ein Warnsignal aus, bei schlechten Bedingungen. Es überprüft erst, ob der CO_2-Gehalt zu hoch ist, dann wird noch geprüft, ob der Buzzer nicht gestummt ist. Wenn der Buzzer nicht gestummt ist, läutet der Buzzer mit einer bestimmten Frequenz, Dauer und Lautstärke. Danach überprüft es, ob der Knopf geklickt wurde, damit der Buzzer gestummt und ausgeschalten werden kann. Bei einem CO_2-Gehalt unter dem gewählten Wert, wird das Stummen aufgehoben.

```
void timerBuzz()
{
  buzzer.configureBuzzer(1000, 1000, SFE_QWIIC_BUZZER_VOLUME_MIN);
  buzzer.on();
  buzzerBuzzing = true;

  if (button.hasBeenClicked() || swipeDown() || swipeLeft() || swipeRight() || swipeUp())
  {
    buzzer.off();
    buzzerBuzzing = false;
    buzzerMuted = true;
    button.clearEventBits();
  }
}
```

Abbildung 29 - timerBuzz()

Die Funktion timerBuzz(), wird für den Timer verwendet. Als erstes, werden die Konfigurationen des Buzzers gesetzt (Frequenz, Dauer, Lautstärke). Danach startet der Buzzer und die Variable buzzerBuzzing wird auf true gesetzt. Falls eine Joystick Bewegung oder ein Knopf klick erkannt wird, stoppt der Timer, buzzerBuzzing wird auf false gesetzt, und buzzerMuted wird auf true gesetzt.

1.4.2.3 Network

```
int wifiTryCount = 0;

// Funktion für die Verbindung mit dem WLAN
void connectWifi()
{
    wifiTryCount = 0;
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.print("Connecting to WiFi...");
    while (WiFi.status() != WL_CONNECTED && wifiTryCount < 20) {
        delay(500);
        Serial.print(".");
        wifiTryCount++;
    }
    if (WiFi.status() == WL_CONNECTED)
    {
        Serial.println("");
        Serial.println("Wifi Connected");
        Serial.println(WiFi.localIP());
        Serial.println(WiFi.getHostname());
        Serial.println("");
    }
    else
    {
        Serial.println("");
        Serial.print("Connection Failed");
        Serial.println("");
    }
}
```

Abbildung 30 - connectWifi()

Die Funktion connectWifi() ist zuständig für die Verbindung mit dem Wlan. Erst wird eine Variable für die Anzahl Versuche der Verbindung deklariert und initialisiert. Danach beginnt der Versuch der Verbindung mit der SSID und dem Password, während der WLAN-Status nicht verbunden ist oder Während der wifiTryCount unter 20 ist.

Wenn die Verbindung aufgebaut wurde, wird das im Serial Monitor ausgegeben, folgend mit der IP-Adresse und dem Namen des Gerätes.

Wenn die Verbindung fehlschlägt, wird das ebenfalls im Serial Monitor ausgegeben.

1.4.2.4 Menu

```
void buttonReleaseHandler()
{
  if (!button.isPressedQueueEmpty() && buttonRelease == true)
  {
    buttonRelease = !buttonRelease;
    button.popPressedQueue();
  }
}
```

Abbildung 31 - buttonReleaseHandler()

Die Funktion buttonReleaseHandler() prüft als erstes, ob es einen Klick in der Warteschlange (Queue) hat und ob der Knopf losgelassen wurde bzw. der Knopf bereit für den nächsten Klick ist. Wenn beide Bedingungen erfüllt sind, wird der buttonRelease gekehrt (der Knopf ist nicht bereit für den nächsten Klick). Der Klick wird anschliessend gelöscht mit popPressedQueue(), damit dieser nicht erneut für die Bedingung verarbeitet wird.

```
void handleMenuChange(menuState newState)
{
  if (!button.isPressedQueueEmpty() && buttonRelease == false)
  {
    currentState = newState;
    buttonRelease = !buttonRelease;
    button.popPressedQueue();
  }
}
```

Abbildung 32 - HandleMenuChange()

Die handleMenuChange() Funktion prüft, ob es einen Klick in der Warteschlange (Queue) hat und ob der Knopf nicht losgelassen wurde bzw. der Knopf nicht für den nächsten Klick bereit ist. Wenn beide Bedingungen erfüllt sind, wird der momentane Zustand (Anzeige) auf einen neuen (ausgewählten) Zustand gesetzt. Der ButtonRelease wird erneut gekehrt (Knopf ist bereit für den nächsten Klick) und der Klick anschliessend gelöscht mit popPressedQueue().

```
void manageMenu()
{
  switch (currentState)
  {
  case CLOCK_STATE:
  {
    handleMenuChange(AIR_QUALITY_STATE);
    buttonReleaseHandler();

    clearDisplayOnce();
    printTimeAndDate();
    break;
  }

  case AIR_QUALITY_STATE:
  {
    handleMenuChange(TIMER_STATE);
    buttonReleaseHandler();

    clearDisplayOnce();
    printTempAndCO2();
    break;
  }
  case TIMER_STATE:
  {
    handleMenuChange(CLOCK_STATE);
    buttonReleaseHandler();

    clearDisplayOnce();
    showTimer();
    chooseOption();
    setTimer();
    swipeUp();
    startTimer();
    break;
  }
  }
}
```

Abbildung 33 - manageMenu()

Die Funktion manageMenu() ist für das Wechseln der Menüs zuständig. Sie arbeitet, indem sie die Funktionen handleMenuChange() und buttonReleaseHandler() kombiniert und anschliessend den entsprechenden Inhalt ausgibt.

Es wird erst geprüft, ob der Knopf geklickt wurde. Wenn dieser geklickt, ändert sich der Zustand.

Zusätzlich wird noch buttonReleaseHandler() ausgeführt, damit die restlichen Tastendrücke aus der Queue entfernt werden und setzt den Knopf zurück auf "bereit", damit der nächste Tastendruck wieder funktioniert.

```
void clearDisplayOnce()
{
  if (lastState != currentState)
  {
    lcd.clear();
    lastState = currentState;
  }
}
```

Abbildung 34 - clearDisplayOnce()

Diese Funktion wird bei jedem Wechsel der Menüs einmal ausgeführt, dazu wird überprüft, ob der letzte Zustand nicht der aktuelle Zustand ist. Falls dies erfüllt wird, wird die Anzeige gelöscht und der letzte Zustand wird auf den aktuellen Zustand gesetzt.

1.4.2.5 Timer

```
void timerInit()
{
    arrowPositionTimer.start(250);
    valueChangeTimer.start(500);
    timerCountdown.start(1000);
}
```

Abbildung 35 - timerInit()

Die Funktion timerInit() ist wird im Setup aufgerufen und wird verwendet, um alle Timer bezüglich der Intervalle der verschiedenen Codes zu starten

```
void initializeTimerVariables()
{
    hourStr = hour < 10 ? "0" + String(hour) : String(hour);
    minuteStr = minute < 10 ? "0" + String(minute) : String(minute);
    secondStr = second < 10 ? "0" + String(second) : String(second);
}
```

Abbildung 36 - InitializeTimerVariables()

Die Funktion initializeTimerVariables(), wird verwendet, damit die Stunden, Minuten und Sekunden mit dem Datentyp «String» aufrufbar sind.

```
bool swipeRight()
{
    if (readAxisX > upRight && valueChangeTimer.isTimeout())
    {
        Serial.println("Swiped Right");
        valueChangeTimer.restart();
        return true;
    }
    return false;
}
```

Die Funktion swipeRight, swipeLeft, swipeUp und swipeDown funktionieren alle sehr ähnlich. Es wird überprüft, ob die bestimmte Position für das Erkennen der Bewegung erreicht wurde. Wenn die Bedingung erfüllt wird, wird ein true ausgeben mit einer Ausgabe im Serial Monitor.

Die Funktion showTimer() zeigt die aktuelle Uhrzeit an, sowie einen Pfeil. Die Position des kann man mit der Funktion chooseOption() einstellen. Bei dem Wert von 7, wird ein Start Symbol angezeigt, welches das Starten des Timers ermöglicht.

```
void showTimer()
{
    if (displayTimer.isTimeout())
    {
        lcd.setCursor(0, 0);
        lcd.print("Timer: ");
        lcd.print(hourStr);
        lcd.print(":");
        lcd.print(minuteStr);
        lcd.print(":");
        lcd.print(secondStr);
        lcd.print(" ");
        lcd.setCursor(arrowPosition, 1);
        if (arrowPosition >= 7)
        {
            lcd.writeChar(4);
        }
        lcd.setCursor(2, 1);
        lcd.writeChar(5);
        displayTimer.restart();
    }
}
```

Abbildung 37 - showTimer()

```
void chooseOption()
{
    bool leftDetected = false;
    bool rightDetected = false;
    if (arrowPositionTimer.isTimeout())
    {
        rightDetected = swipeRight();
        leftDetected = swipeLeft();

        if (rightDetected && arrowPosition < 14 && arrowPosition >= 7)
        {
            do
            {
                arrowPosition += 1;
            } while ((arrowPosition == 9 || arrowPosition == 12) && arrowPosition < 14);
            lcd.clear();
            Serial.println(arrowPosition);
        }
        else if (leftDetected && arrowPosition > 7)
        {
            do
            {
                arrowPosition -= 1;
            } while ((arrowPosition == 9 || arrowPosition == 12));
            lcd.clear();
            Serial.println(arrowPosition);
        }
        arrowPositionTimer.restart();
    }

    if (arrowPosition == 7 && leftDetected)
    {
        arrowPosition = 1;
        lcd.clear();
        lcd.setCursor(2, 1);
    }
    else if (arrowPosition < 7 && rightDetected)
    {
        arrowPosition = 7;
        lcd.clear();
    }
}
```

Abbildung 38 - chooseOption()

Die Funktion chooseOption() wird verwendet für das Einstellen der Position des Pfeilsymboles. Es wird geprüft ob eine Links oder Rechtsbewegung ausgeführt wurde und sichergestellt, dass sich das Pfeilsymbol im begrenzten Bereich liegt. Bei den Werten 9 und 12 wird die Position doppelt addiert bzw. subtrahiert, da dort die Doppelpunkte der Uhrzeit liegen.

```
void setTimer()
{
    switch (arrowPosition)
    {
    case 7:
        if (swipeUp())
        {
            hour += 10;
            if (hour > 99)
                hour = 99;
        }
        else if (swipeDown())
        {
            hour -= 10;
            if (hour < 0)
                hour = 0;
        }

        break;
```

Abbildung 39 - setTimer()

Die Funktion setTimer() ermöglicht das Einstellen des Timers. Dies wird mit Hilfe einer Switch-Case ausgeführt. Je nach Position des Pfeiles sind die Inkremente und die Masseinheiten unterschiedlich. Bei jeder einstellbarer Stelle, wird geprüft, ob ein Wischen nach oben oder nach unten erfolgt wurde. Als Beispiel, wenn man den Joystick nach oben wischt mit der arrowPosition = 7, wird die Stunden Anzahl um 10 erhöht. Falls der Wert nach dem Addieren über den Grenzwert steigt, wird dieser somit auf den Grenzwert gesetzt.

(Auf dem Bild ist nicht die gesamte Funktion zu sehen)

```
void startTimer()
{
    if (timerCountdown.isTimeout())
    {

        if (arrowPosition == 1 && swipedUp && (hour > 0 || minute > 0 || second > 0))
        {
            timerHasStarted = true;

            if (second > 0)
            {
                second--;
            }
            else if (minute > 0)
            {
                minute--;
                second = 59;
            }
            else if (hour > 0)
            {
                hour--;
                minute = 59;
                second = 59;
            }
        }
```

Abbildung 40 - startTimer() Teil 1

Die Funktion startTimer() ermöglicht das Starten des Timers. Als erstes wird überprüft, ob die arrowPosition = 1 ist (beim Start Icon auf dem lcd). Zusätzlich muss man mit dem Joystick nach oben wischen und der Timer muss auch einen Wert haben. Falls alle Bedingungen erfüllt werden, wird die Variable timerHasStarted auf true gesetzt. Wenn die Variable second einen Wert über 0 hat, wird sie diese um 1 subtrahiert, bis es 0 ist. Danach wird überprüft, ob die Variable minute einen Wert über 0 hat, dem entsprechen wird diese um 1 subtrahiert und die Variabel second wird auf 59 gesetzt. Das gleiche passiert mit der Stunde, da werden jedoch beide Variablen second und minute auf 59 gesetzt.

```
        else if (arrowPosition == 1 && swipedUp && hour == 0 && minute == 0 && second == 0 && timerHasStarted)
        {
            button.clearEventBits();
            timerBuzz();
            timerHasStarted = false;
            currentState = TIMER_STATE;
        }
        timerCountdown.restart();
    }
}
```

Abbildung 41 - startTimer() Teil 2

Falls der Timer keinen Wert hat und die anderen Bedingungen nicht erfüllt werden, wird die zweite If-Bedingung ausgeführt. Als erstes werden die Daten vom Knopf gelöscht. Danach wird die Funktion timerBuzz ausgeführt, die variable buzzerBuzzing auf true und der Zustand wird auf den Timer gesetzt, damit sich das Menü auf den Timer ändert.

1.4.2.6 Main

```
void setup()
{
  pinMode(JoyStick_X_Pin, INPUT);
  pinMode(JoyStick_Y_Pin, INPUT);

  Serial.begin(115200);
  Wire.begin();
  timerInit();
  connectWifi();
  timeClient.begin();
  analogReadResolution(7);

  if (!ens160.begin())
  {
    Serial.println("ENS160 Hat nicht geantwortet.");
    while (1)
      ;
  }
```

Abbildung 42 - setup() Teil 1

In der Funktion setup(), werden Initialisierungen von Einstellungen ausgeführt. Mit pinMode() können die Pins, bei dem der Joystick verbunden ist, als Input eingestellt werden. Serial.begin(), ermöglicht den Datenaustausch mit den Computer mit der angegebenen Baudrate. Wire.begin() startet die Kommunikation mit den I²C verbundenen Komponenten. Die Funktion timerInit() startet, alle nötigen Timer, für die “Refresh Rate“ der Anzeige. connectWifi() wird ausgeführt, um die Verbindung mit dem WLAN zu erstellen. timeClient.begin() startet mit dem Aufruf der aktuellen Zeit und initialisiert den Start. analogReadResolution(), setzt die Bitrate, der Position des Joysticks. Desto höher der Wert, desto genauer die Position.

Die Letzte If-Schleife, wird für alle Sensoren, den Buzzer und den Knopf verwendet. Das Komponent wird gestartet, falls das fehlschlägt, gibt es eine Ausgabe und wird gestoppt.

```
  if (ens160.setOperatingMode(SFE_ENS160_RESET))
    Serial.println("Ready.");

  ens160.setOperatingMode(SFE_ENS160_STANDARD);

  lcdSetup();
  button.LEDoff();
}
```

Abbildung 43 - setup() Teil 2

Die If-Schleife setzt den Sensor ens160 zum «Reset-Mode», um alle Zustände und daten zu löschen. Wenn dies gelingt, gibt es eine Ausgabe. Danach wird der ens160 in den «Standard-Mode» gesetzt.

Mit der Funktion lcdSetup() werden alle Konfigurationen des lcds initialisiert. Der LED vom Knopf wird ausgeschaltet.

```
void loop()
{
  // Joystick Werte einlesen
  readAxisY = analogRead(JoyStick_Y_Pin);
  readAxisX = analogRead(JoyStick_X_Pin);

  // NTP Client updaten
  timeClient.update();

  // CO2 Wert und Temperatur auslesen
  ppm = ens160.getECO2();
  temp = bme280.readTempC() - tempDiff;

  // Menü Funktion ausführen
  manageMenu();

  // Bei schlechter Luftbedingung warnen
  warnBuzz();

  // LED bei verwendung des Knopfes anschalten
  button.isPressed() ? button.LEDon(brightness) : button.LEDoff();
}
```

Abbildung 44 - loop()

In der loop() Funktion, werden alle erstellten Funktion, Variablen und jegliche erstellte Code Ausschnitte verwendet, um das Projekt Laufen zu bringen.

Erst werden die X und Y Positionen vom Joystick in eine separate Variable gesetzt. Der timeClient wird immer wieder aktualisiert, damit die richtige Uhrzeit angezeigt wird. Die Temperatur und der ppm Wert vom CO_2 wird ebenfalls in eine separate Variable gesetzt. Danach wird die Funktion manageMenu() aufgerufen. Diese Funktion übernimmt alle Funktionen für die einzelne Menüs. Die Funktion warnBuzz() wird ebenfalls aufgerufen, um den Nutzer bei schlechten Luftbedingungen zu warnen. Der Bedingungsoperator, schaltet den LED vom Knopf an, falls dieser gedruckt wird.

2 Benutzerhandbuch

2.1 Installation

Die Komponente, sollten mit dem I²C Kommunikationsprotokoll arbeiten, sowie muss sichergestellt werden, dass der Pin für den Joystick richtig definiert wurde.

Falls die Verkabelung zwischen den Komponenten fertig ist, muss ein Projekt in der IDE durch PlatformIO erstellt werden. Darin wird der Code hinzugefügt.

Sollte sich der Code nun im Projekt befinden, muss der PC / Laptop mit dem Mikrocontroller verbunden werden. Das gelingt mit der USB-C Schnittstelle am Mikrokontroller. Der Code muss dann hochgeladen werden.

2.2 Allgemeine Steuerungen

Auf der SmartClock gibt es 2 Optionen, um etwas einzustellen. Die erste Variante ist der Knopf. Dieser wird hauptsächlich verwendet, um die Menüs bzw. die Anzeige zu ändern, Sowie um den Buzzer zu stoppen. Die zweite Option ist der Joystick. Der Joystick, übernimmt alle Steuerungen des Timers.

2.2.1 Knopf Steuerung

Wie bereits erwähnt wird der Knopf für das Ändern der Menüs und das Stoppen des Buzzers verwendet. Beim Klicken des Knopfes leuchtet der Knopf, als Bestätigung, dass ein Klick gelungen ist.

2.2.2 Joystick Steuerung

Der Joystick wird für die Steuerung im Timer Menü verwendet. Auf dem Joystick sind folgende Bewegungen möglich: Das wischen nach oben, unten, links und rechts. Das hoch und runter wischen, wird jeweils verwendet, um die Anzahl zu erhöhen bzw. zu reduzieren. Das Wischen nach links und rechts, wird verwendet, um die Position zu ändern.

Abbildung 45 - Timer Positionen

Wie auf dem Bild zu erkennen ist gibt es 7 verschiedene Positionen, auf dem sich der „Cursor“ positionieren kann. Auf den Positionen 2-7 ist das Einstellen des Timers möglich. Jeweils auf der ersten Ziffer der Sekunde, Minute oder Stunde wird die Anzahl um 1 erhöht / reduziert. Auf der zweiten Ziffer wird die Anzahl um 5 erhöht / reduziert. Bei der Position 1. Ist das Starten des Timers möglich. Der Timer startet, wenn das Wischen nach oben gelingt. Dieser kann bei jeder anderen Joystick Bewegung gestoppt werden.

2.3 Wichtige Informationen

2.3.1 Sensoren

Für genaue Messungen ist es empfehlenswert, das Gerät entfernt von Fenstern und vor jeglichen kalten oder warmen Objekten zu platzieren.

2.3.2 Joystick Verbindung

Damit der Joystick verwendet werden kann, muss dieser mit 4 Jumper Wires gelötet werden.

2.3.3 WLAN-Verbindung

Damit die Anzeige der Uhrzeit funktioniert, benötigt man eine WLAN-Verbindung. Dafür kann eine Bibliothek erstellt werden, mit der gegebenen SSID und dem Passwort.

3 Verzeichnis

3.1 Abbildung

Abbildung 1 - MicroMod ESP32 3
Abbildung 2 - MicroMod ATP Carrier Board 3
Abbildung 3 - Environmental Combo Breakout 3
Abbildung 4 - SerLCD 3
Abbildung 5 - Qwiic Button 4
Abbildung 6 - Sparkfun Buzzer 4
Abbildung 7 - Qwiic Cable 4
Abbildung 8 - Multiport 4
Abbildung 9 - Thumb Slide Joystick 5
Abbildung 10 - USB-a zu USB-c 5
Abbildung 11 - Verkabelung V1 6
Abbildung 12 - Verkabelung V2 6
Abbildung 13 - WarnBuzz V1 7
Abbildung 14 - WarnBuzz V2 7
Abbildung 15 - WarnBuzz V3 7
Abbildung 16 - SetTimer V1 8
Abbildung 17 - SetTimer V2 8
Abbildung 18 - SetTimer V3 8
Abbildung 19 - Luftqualität 9
Abbildung 20 - Datum & Uhrzeit 9
Abbildung 21 - Timer 9
Abbildung 22 - Stückliste V1 10
Abbildung 23 - Stückliste V2 10
Abbildung 24 - Zusammenbau 11
Abbildung 25 - lcdSetup() 12
Abbildung 26 - printTimeAndDate() 12
Abbildung 27 - printTempandCO2 12
Abbildung 28 - warnBuzz() 13
Abbildung 29 - timerBuzz() 13
Abbildung 30 - connectWifi() 13
Abbildung 31 - buttonReleaseHandler() 14
Abbildung 32 - HandleMenuChange() 14
Abbildung 33 - manageMenu() 14
Abbildung 34 - clearDisplayOnce() 15
Abbildung 35 - timerInit() 15
Abbildung 36 - InitializeTimerVariables() 15
Abbildung 37 - showTimer() 15
Abbildung 38 - chooseOption() 16
Abbildung 39 - setTimer() 16
Abbildung 40 - startTimer() Teil 1 17
Abbildung 41 - startTimer() Teil 2 17
Abbildung 42 - setup() Teil 1 18
Abbildung 43 - setup() Teil 2 18
Abbildung 44 - loop() 18
Abbildung 45 - Timer Positionen 19